

Aave Labs - Stable Vaults - Extension 2

Final report

Prepared by: Josselin Feist (josselin@seceureka.com)

July 6th, 2026

Prepared For: Aave Labs

Executive Summary	3
Overview	3
Security findings	4
aggregate3Value accumulator overflow can allow spending Ether already held by OwnedMulticall	5
Summary	5
Recommendations	6
Status	6
Appendix E: April 27th review result	7
Bridging	7
StableVault	7
General	8

Executive Summary

Overview

During the week of April 27, we performed a one-week review of the Stable Vault contracts, covering the changes made to the contracts since the last review. We focused on the changes made to the src directory:

- Commit `b0e5db5b90c1c008f8a1eeeb68c1e9bffe40d134` (April 27th)
- Commit `9aa415cf52eeef829a757f698d8ed38aef86f146` (April 28th)
- Commit `c9ee565c74408e1a79162616cfcdc6553b5f329d` (April 29th - May 1st)

The results of our review are presented in [Appendix A](#).

This report is part of a multiple stages review of the Stable Vault (3/4).

Security findings

ID	Title	Severity	Impact	Likelihood	Status
01	aggregate3Value accumulator overflow can allow spending Ether already held by OwnedMulticall	<i>Informational</i>	<i>Informational</i>	<i>Informational</i>	<i>Acknowledged</i>

aggregate3Value accumulator overflow can allow spending Ether already held by OwnedMulticall

*Impact:
Informational*

Likelihood: Low

Status: Acknowledged

001

Summary

aggregate3Value is supposed to only spend the native token supplied through `msg.value` for the current multicall. It enforces this at the end with `msg.value == valAccumulator`.

However, the accumulator is updated in an unchecked block before knowing whether the low-level call succeeded:

```
unchecked {
    valAccumulator += val;
}
(result.success, result.returnData) =
calli.target.call{value: val}(calli.callData);
assembly {
    // Revert if the call fails and failure is not allowed
    // `allowFailure := calldataload(add(calli, 0x20))` and
    `success := mload(result)`
    if iszero(or(calldataload(add(calli, 0x20)),
mload(result))) {
        // set "Error(string)" signature:
        bytes32(bytes4(keccak256("Error(string)")))
        mstore(0x00,
0x08c379a000000000000000000000000000000000000000000000000000000000)
        // set data offset
        mstore(0x04,
0x0000000000000000000000000000000000000000000000000000000000000020)
        // set length of revert string
        mstore(0x24,
0x0000000000000000000000000000000000000000000000000000000000000017)
        // set revert string:
        bytes32(abi.encodePacked("Multicall3: call failed"))
        mstore(0x44,
```

```

0x4d756c746963616c6c333a2063616c6c206661696c6564000000000000000000)
    // NOTE: The original Multicall3 uses 0x84 here (132
bytes) instead of the correct
    // 0x64 (100 bytes), including 32 bytes of dirty
memory in the revert data.
    // Kept to stay consistent with the canonical
Multicall3 source.
    revert(0x00, 0x84)
}
}
unchecked {
    ++i;
}
}
// Finally, make sure the msg.value = SUM(call[0...i].value)
require(msg.value == valAccumulator, "Multicall3: value
mismatch");

```

Figure 1: src/periphery/OwnedMulticall.sol#L183-L210

If a call is allowed to fail, a very large value can wrap valAccumulator while the call itself fails because the contract does not have enough ether.

This can make the final value check pass while a later subcall spends Ether that was already sitting in the contract.

The issue is informational, as no ether is supposed to be in the contract.

Recommendations

Do not count the value of failed calls toward the final accumulator.

We also recommend to consider using a contract other than OwnedMulticall for the rebalancer (see [Appendix E](#))

Status

Aave Labs stated:

Acknowledged; the defect is real but unreachable in the production threat model, so we're keeping the canonical Multicall3 source verbatim.

Appendix E: April 27th review result

The review identifies one vulnerability in OwnedMulticall ([issue 17](#)), as well as code clarity, general improvements and operational tradeoffs that should be made explicit.

The proposed improvements shared with Aave Labs are:

Bridging

- **Clarify the exceptions for blocking sendPushFundsToChainMessage if the L2 sequencer is down.**
 - ChainBalance.isStale currently represents both remote balance data staleness and local L2 sequencer health. Blocking for the latter means that liquidity will not be movable from the AC to the EC if the AC sequencer is down
- **Remove the refund mechanism for the fee with tokens payment.** estimatedFeeAmount can be transferred from the user, instead of bridgeParams.feeAmount, removing the need for the refund mechanism (except when using the native token)
- **Consider not using address(0x0) for both the NATIVE_CURRENCY and ASSET_FOR_DATA_ONLY_BRIDGE.** Using the same value to represent the native asset and data-only message increases complexity and the likelihood of a collision bug.
- **Check that supportsInterface actually returns true in ChainBalanceOracle.** The return value is not checked, and therefore the destination contract might not support the interface
- **Consider calling forceApprove(x, 0) to reset approval in _depositToStrategy and publishMessageToChainWithFeePayer.** This will enforce that the approval is reset, no matter the previous operation behavior.
- **Document Assumption #20 in the contracts' NatSpec.** The assumption: *Native asset refunds from the CCIP bridge adapter will always be sent to an EOA* is likely to be missed by the integrators. Consider making it explicit in
 - pushFundsToAccountingChain
 - exchangeLouTokens
 - bridgeTokens
 - pushFundsToChain

StableVault

- **Update the StableVault Role spreadsheet.** The following inaccuracies were found:
 - Delete setDefaultBridgeAdapter
 - Remove uint8 from Allocator.addStrategy signature
 - Add address bridgeAdapter, bytes bridgeParamsEncoded to the bridge function signatures

- `claimSurplusInterest`'s rationale can be updated with the recent logic
- `setAssetFeeBps` cap is 10% (not 5%) (in the rationale column)
- `CCIPAdapter.rescueTokens` rationale could make it clear that only non registered tokens are rescuable (which is expected)
- Replace BBV with StableVault in the rationale column of L49/50
- **Avoid ERC20-style naming/events for the StableVault.** The vault does not behave like an ERC20 token, and the name might incorrectly lead integrators to assume it does
- **Clarify the amount emitted by the Transfer event.** For example, on deposit, `netDepositAmountInRay.assetDecimalsToRay(asset)` is credited, but the event shows `amount.assetDecimalsToRay(asset)`. It is unclear if the event is meant to show the value as requested, or actually credited. Alternatively, the event could also be expected to show the shares minted.

General

- **Make `MIN_BURN_IOU_TOKEN_GAS_LIMIT` configurable as an immutable constructor parameter instead of a source-code constant.** This will allow its value to be changed without changing the source code
- **Consider not using `OwnedMulticall` and writing a tailored integration contract.** `OwnedMulticall` is meant to be used by the rebalancer, but the contract was not built for user-constrained operations, and requires adaptation to fit Aave Labs' need (ex: there is no support for the contract to hold native assets).
- **Do not rely on KYC as a source of security.** An ongoing investigation of the rate limit mechanism was discussed during the review, and part of the mitigation relied on KYC information. KYC can easily be bypassed by sophisticated attackers, and should not be relied upon for the system's security